

Практическая теория типов

Лекция 8: Субструктурные логики и типы

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

- Естественный вывод представляет «естественный вывод» для рассуждений о выводимости:

$$A_1, \dots, A_n \vdash B$$

- В пропозиционном исчислении допустима запись следующего утверждения

$$p, \neg q \vdash q \Rightarrow (p \Rightarrow r)$$

- Соотношение Карри-Ховарда устанавливает связь между логикой и типами, позволяющую переносить идеи из логики в языки программирования и обратно.

- Правила вывода, связанные с манипуляциями предпосылками (формулами в контексте), называются субструктурными
- Они позволяют рассматривать контекст, как множество формул

$$\text{EXCHANGE} \frac{\Gamma, A, B, \Delta \vdash C}{\Gamma, B, A, \Delta \vdash C}$$

$$\text{CONTRACTION} \frac{\Gamma, A, A, \Delta \vdash B}{\Gamma, A, \Delta \vdash B}$$

$$\text{WEAKENING} \frac{\Gamma, \Delta \vdash B}{\Gamma, A, \Delta \vdash B}$$

- Если какая-то логическая система не использует одно из данных правил вывода, то она называется субструктурной

- Контекст в логических системах соответствует состоянию программы (например, набору переменных и их значений) в определённый момент времени
- Применение идей соответствия Карри-Ховарда позволяет ограничивать операции над переменными, получая различные субструктурные системы типов:

	Exchange	Weakening	Contraction	Use
Ordered	—	—	—	Exactly once in order
Linear	Allowed	—	—	Exactly once
Affine	Allowed	Allowed	—	At most once
Relevant	Allowed	—	Allowed	At least once
Normal	Allowed	Allowed	Allowed	Arbitrarily

План лекции

- 1 Мотивация
- 2 Линейная логика**
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

Линейная логика: основная идея

- Линейная логика (Linear Logic) — логика ресурсов, предложенная Жан-Ивом Жираном в 1987 году
- В линейной логике не доступно ни ослабление, ни контракция
- Каждая формула (ресурс) должна использоваться в точности один раз
- Ключевая идея: различие между:
 - формула A истинна (классическая логика)
 - имеется доступный ресурс A (линейная логика)
- Применения:
 - Управление памятью
 - Параллельные вычисления (process calculus)
 - Сессионные типы (коммуникационные протоколы)

Модальности в логике

Определение модальности

- Модальность — логический оператор, изменяющий режим истинности формулы
- В модальной логике: $\Box A$ (необходимость A), $\Diamond A$ (возможность A)
- В линейной логике: $!A$ (неограниченная доступность A), $?A$ (опциональная доступность A)

Роль модальностей в субструктурных логиках

Модальности позволяют локально восстанавливать структурные правила:

- $!A$ в линейной логике — формула A , для которой разрешены weakening и contraction
- Это позволяет смешивать линейные и обычные формулы в одной системе
- Обычная логика вкладывается в линейную через перевод Жирара

Аналогия: модальности являются типами типов, контролирующими правила работы с формулами

Линейная логика имеет богатую систему связок:

- Мультипликативные связки (разделяют ресурсы):
 - $A \otimes B$ — мультипликативная конъюнкция (tensor)
 - $A \wp B$ — par (дуальная к $\otimes$, в классической LL)
 - $A \multimap B$ — линейная импликация (linear implication)
 - 1 — мультипликативная единица (для $\otimes$)
 - $\perp$ — мультипликативный ноль (для $\wp$, в классической LL)
- Аддитивные связки (выбирают между ресурсами):
 - $A \& B$ — аддитивная конъюнкция (with)
 - $A \oplus B$ — аддитивная дизъюнкция (plus)
 - $\top$ — аддитивная единица (для $\&$)
 - 0 — аддитивный ноль (для $\oplus$)
- Экспоненциальные модальности (восстанавливают структурные правила):
 - $!A$ — модальность утверждения (разрешает копирование и удаление)
 - $?A$ — дуальная модальность

Замечание: в интуиционистской линейной логике (ILL) обычно используют $\otimes, \multimap, \&, \oplus$

Мнемоника (классическая LL): $A \multimap B \equiv A^\perp \wp B$ — линейная импликация через дуальность

Мультипликативные vs аддитивные связки

Разница между $\otimes$ и $\&$:

Мультипликативная $\otimes$

$A \otimes B$ — имеется ресурс A и ресурс B

Ресурсы разделяются:

$$\frac{\Delta_1 \vdash A \quad \Delta_2 \vdash B}{\Delta_1, \Delta_2 \vdash A \otimes B}$$

Пример: композиция двух различных валют соответствует операции tensor

Аддитивная $\&$

$A \& B$ — возможность выбора A или B

Один контекст для обоих:

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B}$$

Пример: выбор между двумя альтернативами из общего набора ресурсов соответствует операции with

Вывод: $\otimes$ требует независимые ресурсы, $\&$ — один набор ресурсов с выбором

Замечание: $\&$ — внешний выбор (контекст выбирает проекцию), $\oplus$ — внутренний (доказывающий выбирает)

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система**
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

Constructive Logic

$$\frac{A \in \Gamma}{\Gamma \vdash A} \text{ (Hyp)}$$

Linear Logic

$$\overline{A \vdash A} \text{ (Hyp)}$$

При наличии единственной формулы A допускается использование исключительно данной формулы без расширения контекста

Линейная логика: линейная импликация

Линейная импликация $A \multimap B$ означает потребление одного экземпляра A и производство одного экземпляра B

Constructive Logic

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B}$$

$$\frac{\Gamma \vdash A \supset B \quad \Gamma \vdash A}{\Gamma \vdash B}$$

Контекст Γ используется дважды

Linear Logic

$$\frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B}$$

$$\frac{\Delta \vdash A \multimap B \quad \Delta' \vdash A}{\Delta, \Delta' \vdash B}$$

Контексты разделены

Vending machine

English

A coin can buy you a piece of candy, a drink, or be consumed.

Logic

$\text{Coin} \multimap \text{Candy}$
 $\text{Coin} \multimap \text{Drink}$
 $\text{Coin} \multimap 1$

Rust

```
fn buy_candy(_ : Coin)
  -> Candy { Candy{} }
fn buy_drink(_ : Coin)
  -> Drink { Drink{} }
```

- В классической и интуиционистской логике является выводимым следующее суждение

$$A \rightarrow B, A \rightarrow C, A \vdash B \wedge C$$

- однако в линейной логике данное суждение не является выводимым в силу правил вывода

$$A \multimap B, A \multimap C, A \not\vdash B \otimes C$$

- требуется наличие двух экземпляров формулы A

$$A \multimap B, A \multimap C, A, A \vdash B \otimes C$$

Constructive Logic

$$\frac{\Gamma \vdash A_1 \quad \Gamma \vdash A_2}{\Gamma \vdash A_1 \wedge A_2} (\wedge I)$$

$$\frac{\Gamma \vdash A_1 \wedge A_2}{\Gamma \vdash A_1} (\wedge E1)$$

$$\frac{\Gamma \vdash A_1 \wedge A_2}{\Gamma \vdash A_2} (\wedge E2)$$

Linear Logic

$$\frac{\Delta_1 \vdash A_1 \quad \Delta_2 \vdash A_2}{\Delta_1, \Delta_2 \vdash A_1 \otimes A_2} (\otimes I)$$

$$\frac{\Delta \vdash A_1 \otimes A_2 \quad \Delta', A_1, A_2 \vdash C}{\Delta, \Delta' \vdash C} (\otimes E)$$

Линейная логика: дизъюнкция

Constructive Logic

$$\frac{\Gamma \vdash A_1}{\Gamma \vdash A_1 \vee A_2} (\vee I_1) \qquad \frac{\Gamma \vdash A_2}{\Gamma \vdash A_1 \vee A_2} (\vee I_2)$$

$$\frac{\Gamma \vdash A_1 \vee A_2 \quad \Gamma, A_1 \vdash B \quad \Gamma, A_2 \vdash B}{\Gamma \vdash B} (\vee E)$$

Linear Logic

$$\frac{\Delta \vdash A_1}{\Delta \vdash A_1 \oplus A_2} (\oplus I_1) \qquad \frac{\Delta \vdash A_2}{\Delta \vdash A_1 \oplus A_2} (\oplus I_2)$$

$$\frac{\Delta \vdash A_1 \oplus A_2 \quad \Delta', A_1 \vdash B \quad \Delta', A_2 \vdash B}{\Delta, \Delta' \vdash B} (\oplus E)$$

Экспоненциальные модальности

Проблема: в чистой линейной логике невозможно выражение обычных формул

Решение: модальности ! и ? восстанавливают структурные правила

Модальность !A

!A обозначает неограниченный ресурс A

Допускает следующие операции:

- Копирование (contraction)
- Удаление (weakening)

$$\frac{!\Gamma \vdash A}{!\Gamma \vdash !A} (!I)$$

где $!\Gamma = !A_1, \dots, !A_n$

Правила для !

Dereliction:

$$\frac{\Gamma, A \vdash B}{\Gamma, !A \vdash B}$$

Contraction для !:

$$\frac{\Gamma, !A, !A \vdash B}{\Gamma, !A \vdash B}$$

Weakening для !:

$$\frac{\Gamma \vdash B}{\Gamma, !A \vdash B}$$

Аддитивная конъюнкция $\&$

Правила:

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} (\&I) \qquad \frac{\Gamma \vdash A \& B}{\Gamma \vdash A} (\&E_1) \qquad \frac{\Gamma \vdash A \& B}{\Gamma \vdash B} (\&E_2)$$

Отличие от $\otimes$

- $\otimes$ — оба ресурса одновременно (независимые контексты)
- $\&$ — выбор одного из двух (один контекст для обоих)

Пример: $A \rightarrow B, A \rightarrow C, A \vdash (B \& C)$ выводимо,
однако $A \multimap B, A \multimap C, A \not\vdash B \otimes C$ (требуется два экземпляра A)

Единицы в линейной логике

Мультипликативная единица 1

Нейтральный элемент для $\otimes$:

$$A \otimes 1 \cong A \cong 1 \otimes A$$

Правила:

$$\overline{\cdot \vdash 1}$$

$$\frac{\Gamma \vdash 1 \quad \Delta \vdash A}{\Gamma, \Delta \vdash A}$$

Пустой ресурс

Замечание: также есть $\perp$ (мультипликативный ноль) и 0 (аддитивный ноль)

Аддитивная единица $\top$

Нейтральный элемент для $\&$:

$$A \& \top \cong A$$

Правила:

$$\overline{\Gamma \vdash \top}$$

Всегда доступный выбор

Дуальность в линейной логике

Линейное отрицание $A^\perp$ — дуальный тип

Ключевые дуальности (классическая LL)

- $(A \otimes B)^\perp = A^\perp \wp B^\perp$ — де Морган для мультипликативных
- $(A \oplus B)^\perp = A^\perp \& B^\perp$ — де Морган для аддитивных
- $(A \multimap B)^\perp = A \otimes B^\perp$ — дуальность функции (если $A \multimap B \equiv A^\perp \wp B$)
- $(!A)^\perp = ?(A^\perp)$ — дуальность экспоненциалов

Применение

Дуальность используется в сессионных типах:
если клиент отправляет, то сервер получает

Свойство (в классической LL): если $\Gamma \vdash A$, то $A^\perp \vdash \Gamma^\perp$

Пример: мини-протокол клиент-сервер

Протокол простого запроса-ответа:

Клиент

- Отправляет Request
- Получает Response
- Завершает сессию

Тип канала:

$$\text{Request} \otimes (\text{Response} \multimap 1)$$

«Отправить запрос, получить ответ»

Сервер (дуальный)

- Получает Request
- Отправляет Response
- Завершает сессию

Тип канала:

$$(\text{Request} \otimes (\text{Response} \multimap 1))^{\perp}$$

«Получить запрос, отправить ответ»

Дуальность гарантирует

Корректная типизация обеспечивает завершение взаимодействия без дедлоков

- Линейные типы — типовая система, соответствующая линейной логике по Карри-Ховарду
- Основная идея: для типизации $\Gamma \vdash e : \tau$ необходимо использовать в точности те же переменные из Γ , что встречаются в e
- Каждая переменная из контекста должна быть использована ровно один раз в терме

Линейные типы

Грамматика для термов

$$e ::= x \mid \lambda x : \tau. e \mid e_1 e_2$$

Грамматика для типов

$$\tau ::= b \mid \tau_1 \multimap \tau_2$$

Правила типизации

$$\frac{}{x : \tau \vdash x : \tau} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1. e : \tau_1 \multimap \tau_2} \quad \frac{\Gamma \vdash e_1 : \tau_1 \multimap \tau_2 \quad \Delta \vdash e_2 : \tau_1}{\Gamma, \Delta \vdash e_1 e_2 : \tau_2}$$

В полученной системе типов не типизируется

$$\lambda f : \text{int} \multimap \text{int}. \lambda x : \text{int}. f (f x)$$

поскольку переменная f используется дважды

Интерпретация экспоненциалов

Вложение интуиционистской логики в линейную:

Перевод Жирара (Girard translation):

$$(A \wedge B)^* = !A^* \otimes !B^*$$

$$(A \vee B)^* = !A^* \oplus !B^*$$

$$(A \Rightarrow B)^* = !A^* \multimap B^*$$

$$(\neg A)^* = !A^* \multimap 0$$

Следствие

Интуиционистская логика = фрагмент линейной логики с ограниченным использованием модальности !

Точнее: суждение $\Gamma \vdash A$ вкладывается как $! \Gamma \vdash A^*$, где $! \Gamma = \{!A_1, \dots, !A_n\}$

Это показывает, что линейная логика обобщает обычную логику, добавляя контроль над ресурсами

Промежуточное резюме: линейная логика

Основные идеи

- Линейная логика = логика ресурсов (ровно один раз)
- $\otimes, \multimap$ — разделяют ресурсы (мультипликативные)
- $\&, \oplus$ — выбирают между ресурсами (аддитивные)
- $!$ — восстанавливает weakening и contraction

Ключевые свойства

- Cut-elimination завершается, обеспечивая консистентность
- Дуальность для коммуникационных протоколов
- Обобщает интуиционистскую логику

Cut-elimination в линейной логике

Теорема (cut-elimination):

Любое доказательство в линейной логике может быть преобразовано к доказательству без правила cut

Соответствие с вычислениями

- Cut-elimination $\leftrightarrow \beta$ -редукция
- Нормальное доказательство $\leftrightarrow$ нормальная форма терма
- Процесс устранения cut $\leftrightarrow$ выполнение программы

Свойства

- Confluence (слияние): порядок редукций не важен
- Strong normalization: все последовательности редукций конечны
- Subject reduction: типы сохраняются при редукции

Вывод: линейная логика вычислима и консистентна

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов**
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

Линейные типы: формальное определение

Определение: каждая переменная используется ровно один раз

Суждение о типизации: $\Gamma \vdash e : \tau$

Основное ограничение

Каждая переменная $x \in \Gamma$ встречается в e ровно один раз

Разделение контекста:

$$\frac{\Gamma_1 \vdash e_1 : \tau_1 \multimap \tau_2 \quad \Gamma_2 \vdash e_2 : \tau_1}{\Gamma_1, \Gamma_2 \vdash e_1 e_2 : \tau_2}$$

где $\Gamma_1 \cap \Gamma_2 = \emptyset$ (непересекающиеся контексты)

Линейные типы: система правил

Типы: $\tau ::= b \mid \tau_1 \multimap \tau_2 \mid \tau_1 \otimes \tau_2 \mid !\tau$

Термы: $e ::= x \mid \lambda x.e \mid e_1 e_2 \mid (e_1, e_2) \mid \text{let } (x, y) = e_1 \text{ in } e_2$

Правила:

$$\frac{}{x : \tau \vdash x : \tau} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.e : \tau_1 \multimap \tau_2} \quad \frac{\Gamma_1 \vdash e_1 : \tau_1 \multimap \tau_2 \quad \Gamma_2 \vdash e_2 : \tau_1}{\Gamma_1, \Gamma_2 \vdash e_1 e_2 : \tau_2}$$

$$\frac{\Gamma_1 \vdash e_1 : \tau_1 \quad \Gamma_2 \vdash e_2 : \tau_2}{\Gamma_1, \Gamma_2 \vdash (e_1, e_2) : \tau_1 \otimes \tau_2}$$

Экспоненциал в линейных типах

Модальность ! восстанавливает структурные правила:

$$\frac{! \Gamma \vdash e : \tau}{! \Gamma \vdash e : ! \tau} \text{ (!I)}$$

Ослабление и контракция:

$$\frac{\Gamma \vdash e : \tau}{\Gamma, x : ! \tau' \vdash e : \tau} \text{ (W)}$$

$$\frac{\Gamma, x : ! \tau, y : ! \tau \vdash e : \tau'}{\Gamma, z : ! \tau \vdash e[z/x, z/y] : \tau'} \text{ (C)}$$

Интерпретация

- $! \tau$ — неограниченный доступ к τ
- Допускает копирование и удаление
- В Rust: трейт Copy

Теоремы о линейных типах

Теорема (Subject reduction / Сохранение типов)

Если $\Gamma \vdash e : \tau$ и $e \rightarrow e'$, то $\Gamma \vdash e' : \tau$

Типы сохраняются при редукции

Теорема (Progress / Прогресс)

Если $\cdot \vdash e : \tau$, то либо e — значение, либо $\exists e'. e \rightarrow e'$

Хорошо типизированные программы не зависят

Теорема (Нормализация)

Если $\Gamma \vdash e : \tau$, то существует конечная последовательность редукций, приводящая e к нормальной форме

Все типизированные программы терминируют (для чистой линейной λ -исчисления)

Соответствие Карри-Ховарда для линейных типов

Логика	Типы	Вычисления
Формула A	Тип τ	Спецификация
Доказательство A	Терм $e : \tau$	Программа
$A \multimap B$	$\tau_1 \multimap \tau_2$	Линейная функция
$A \otimes B$	$\tau_1 \otimes \tau_2$	Линейная пара
$!A$	$!\tau$	Неограниченный ресурс
Cut-elimination	β -редукция	Выполнение программы
Нормализация	Терминация	Завершение программы

Расширенное соответствие

Линейная логика обобщает интуиционистскую логику (через перевод Жирара):

- $(A \Rightarrow B)^* = !A^* \multimap B^*$
- $(A \wedge B)^* = !A^* \otimes !B^*$
- Обычное λ -исчисление вкладывается в линейное через $!$

Управление памятью: проблема

Классические подходы к управлению памятью:

Manual memory management (C/C++)

```
int* p = malloc(sizeof(int));
*p = 42;
free(p);
*p = 13; // use-after-free

int* q = malloc(sizeof(int));
free(q);
free(q); // double-free
```

Проблемы: use-after-free, double-free, memory leaks

Решение: линейные/аффинные типы дают гарантии безопасности без GC

Garbage collection (Java/Python)

```
Object obj = new Object();
obj.doSomething();
// GC will collect obj later
```

Проблемы:

- Непредсказуемые паузы
- Накладные расходы
- Нет контроля над деструкторами

Линейные типы в Rust: ownership

Rust использует аффинные типы (линейные + weakening):

```
fn main() {  
    let s1 = String::from("hello");  
    let s2 = s1; // s1 moved to s2  
  
    println!("{}", s2); // OK  
    // println!("{}", s1); // ERROR: s1 no longer owns  
}
```

Интерпретация:

- String — линейный тип (нельзя копировать)
- Присваивание $s2 = s1$ — перемещение владения (move)
- После move переменная $s1$ больше не типизируется

$$\Gamma, x : \tau \vdash e : \sigma$$

- Соответствие: $\Gamma \vdash \text{let } y = x \text{ in } e : \sigma$ (где x исчезает)

Линейные типы в Rust: borrowing

Заимствование (borrowing) — временное использование без передачи владения:

```
fn print_length(s: &String) {  
    println!("Length: {}", s.len());  
}  
  
fn main() {  
    let s1 = String::from("hello");  
    print_length(&s1);  
    println!("{}", s1); // s1 still owns  
}
```

Логическая интерпретация:

- $\&T$ — заимствованная ссылка (не владеющая)
- Заимствование $\approx$ модальность $!$ в линейной логике
- $\&T$ может копироваться (несколько неизменяемых ссылок)
- Правила гарантируют отсутствие алиасинга с мутацией

Замечание: строгая формализация требует рассмотрения capabilities

Пример: файловый дескриптор

Линейные типы гарантируют корректное закрытие ресурсов:

```
impl File {  
    fn write(self, data: &[u8]) -> File { /* ... */ }  
    fn close(self) { /* release resource */ }  
}  
  
fn main() {  
    let f = File::open("data.txt");  
    let f = f.write(b"hello");  
    f.close(); // guaranteed close  
    // f.write(b"world"); // ERROR: closed  
}
```

- write потребляет и возвращает File
- close потребляет файл навсегда
- Повторное использование → ошибка компиляции

Сессионные типы: введение

Сессионные типы — применение линейной логики к коммуникации процессов

- Тип описывает протокол взаимодействия по каналу
- Линейность канала гарантирует:
 - Каждое сообщение отправляется/принимается ровно один раз
 - Невозможность дублирования/потери сообщений
 - Отсутствие дедлоков (при правильной типизации)
- Основные конструкции:
 - $\text{send } T.S$ — отправить T , продолжить с сессией S
 - $\text{recv } T.S$ — принять T , продолжить с сессией S
 - $S_1 \oplus S_2$ — выбрать и выполнить S_1 или S_2
 - $S_1 \& S_2$ — предложить выбор между S_1 и S_2
 - end — завершить сессию

Соответствие: сессионные типы $\cong$ формулы линейной логики

Грамматика сессионных типов:

$$S ::= \text{send } \tau. S \mid \text{recv } \tau. S \mid \oplus \{l_i : S_i\} \mid \& \{l_i : S_i\} \mid \text{end}$$

Замечание: `send` и `recv` — конструкторы сессионных типов, отличные от экспоненциалов `!` и `?` в LL

Конструкции

- `send  $\tau. S$` — отправка
- `recv  $\tau. S$` — получение
- $\oplus$ — выбор
- $\&$ — предложение
- `end` — завершение

Дуальность

- $(\text{send } \tau. S)^\perp = \text{recv } \tau. S^\perp$
- $(\oplus \{l_i : S_i\})^\perp = \& \{l_i : S_i^\perp\}$

Клиент и сервер имеют дуальные типы

Связь: `send  $\tau. S$` соответствует $\tau \otimes S$, `recv  $\tau. S$` соответствует $\tau \multimap S$

Сессионные типы: гарантии безопасности

Гарантии системы типов

- Session fidelity: процесс следует протоколу
- Deadlock freedom: нет дедлоков при стандартных ограничениях*
- Type safety: нет несоответствия типов сообщений
- Progress: процессы не зависают

Соответствие с логикой

- Канал = линейный ресурс (используется ровно один раз)
- Cut = порождение канала + параллельная композиция
- Cut-elimination = коммуникационные редукции (выполнение протокола)

Применение: Links, Singularity OS, Rust session types

*Deadlock freedom обеспечивается при линейности каналов, дуальности концов и отсутствии небезопасного разделения

Сессионные типы: пример

Протокол калькулятора:

Клиент отправляет два числа, сервер возвращает сумму:

Тип сессии (клиент):

$$S_{\text{client}} = \text{send Int. send Int. recv Int. end}$$

Отправить значение типа int, отправить значение типа int, принять значение типа int, завершить сессию

```
send(chan, 5);  
send(chan, 3);  
let result = recv(chan);  
close(chan);
```

Тип сессии (сервер):

$$S_{\text{server}} = \text{recv Int. recv Int. send Int. end}$$

Принять значение типа int, принять значение типа int, отправить значение типа int, завершить сессию

```
let x = recv(chan);  
let y = recv(chan);  
send(chan, x + y);  
close(chan);
```

Дуальность: $S_{\text{server}} = S_{\text{client}}^{\perp}$ (в линейной логике)

Сессионные типы: выбор

Внутренний выбор ($\oplus$) vs внешний выбор ($\&$):

Внутренний выбор $\oplus$

Процесс сам выбирает и отправляет метку:

$$S = \oplus \left\{ \begin{array}{l} \text{add} : \text{send Int. send Int. recv Int. end} \\ \text{mul} : \text{send Int. send Int. recv Int. end} \end{array} \right.$$

Клиент выбирает операцию и отправляет метку (add или mul)

Соответствует $\oplus$ в линейной логике

Дуальность: $(\oplus \{l_i : S_i\})^\perp = \& \{l_i : S_i^\perp\}$

Внешний выбор $\&$

Процесс ожидает метку и ветвится:

$$S = \& \left\{ \begin{array}{l} \text{ok} : \text{recv Int. end} \\ \text{error} : \text{recv String. end} \end{array} \right.$$

Клиент ожидает метку от сервера (ok или error) и реагирует

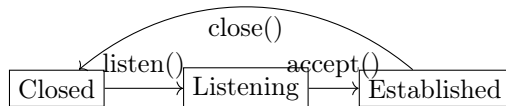
Соответствует $\&$ в линейной логике

Typestate programming

Typestate — паттерн, где тип объекта отражает его текущее состояние:

- Классический ООП: объект имеет один тип, состояние хранится в полях
- Typestate: каждое состояние — отдельный тип

Пример: TCP соединение



- Каждое состояние — тип: `Socket<Closed>`, `Socket<Listening>`, ...
- Методы потребляют объект и возвращают новый тип:
`fn listen(self: Socket<Closed>) -> Socket<Listening>`
- Линейность гарантирует: старое состояние больше недоступно

Typestate: пример на Rust

```
struct Socket<State> { fd: i32, ... }

impl Socket<Closed> {
    fn listen(self) -> Socket<Listening> { ... }
}

impl Socket<Listening> {
    fn accept(self) -> Socket<Established> { ... }
}

impl Socket<Established> {
    fn send(self, data: &[u8])
        -> Socket<Established> { ... }
    fn close(self) -> Socket<Closed> { ... }
}
```

Гарантии: вызов метода `send` на закрытом сокете является невозможным

Параллельное программирование

Линейные типы помогают избежать data races:

Data race — одновременный доступ к памяти из разных потоков, где хотя бы один — запись

Подход Rust

- Send: передача в другой поток; Sync: разделение между потоками
- Mutex<T>: эксклюзивный доступ; Arc<T>: разделяемое владение

Линейная интерпретация

- Передача данных в поток $\equiv$ move (линейность)
- Mutex::lock() возвращает MutexGuard — линейный тип
- При выходе из области MutexGuard автоматически разблокируется

Другие применения линейных типов

Квантовые вычисления

- No-cloning theorem
- Языки: Quipper, Q#

Linear Haskell

- In-place обновления
- Streaming без GC

Базы данных

- Транзакции как ресурсы
- Гарантия commit/rollback

Встроенные системы

- Управление GPIO, таймерами
- Корректная инициализация

Общее: линейность гарантирует корректное управление ресурсами

Промежуточное резюме: линейные типы

Метатеоретические свойства

- Soundness: типизированные программы безопасны
- Strong normalization: все вычисления конечны
- Confluence: порядок редукций не важен

Практические гарантии

- Управление временем жизни ресурсов
- Протоколы использования API
- Отсутствие алиасинга с мутацией
- Корректность коммуникационных протоколов

Ключевая идея: Линейность = контроль использования через систему типов

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика**
- 6 Применения субструктурных типов
- 7 Заключение

Аффинная логика: определение

- Аффинная логика — субструктурная логика, допускающая weakening, но не contraction
- Каждая формула может использоваться не более одного раза:
 - Допускается неиспользование ресурса (weakening)
 - Дублирование ресурса запрещено (no contraction)
- Получается из линейной логики добавлением правила weakening:

$$\frac{\Gamma \vdash B}{\Gamma, A \vdash B} \text{ (Weak)}$$

Интуиция

Уникальное владение ресурсом означает, что владелец может использовать или уничтожить ресурс, но не скопировать его

Правила вывода аффинной логики

Аффинная логика наследует правила линейной логики + добавляет weakening

Основные правила:

$$\frac{}{A \vdash A} \text{ (Hyp)} \qquad \frac{\Gamma \vdash B}{\Gamma, A \vdash B} \text{ (Weak)}$$

Импликация (та же, что в линейной логике):

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \text{ } (\multimap \text{ I}) \qquad \frac{\Gamma \vdash A \multimap B \quad \Delta \vdash A}{\Gamma, \Delta \vdash B} \text{ } (\multimap \text{ E})$$

Ключевое отличие от линейной логики:

- В линейной: все формулы в Γ должны быть использованы
- В аффинной: формулы в Γ могут быть проигнорированы (weakening)

Связь аффинной и линейной логики

Вложение линейной логики в аффинную: тривиально (аффинная = линейная + weakening)

Вложение аффинной логики в линейную:

Прямой подход

Аффинная логика = линейная логика + правило weakening

Для моделирования «только weakening» в чистой LL используют расширения (subexponentials, graded modalities) либо просто добавляют weakening на уровне правил

Аффинная логика и квантовые вычисления

Применение: квантовые вычисления естественно моделируются аффинными типами

- No-cloning theorem: квантовое состояние нельзя скопировать

$$|\psi\rangle \not\rightarrow |\psi\rangle \otimes |\psi\rangle$$

- Измерение разрушает состояние: после измерения квантовый бит становится классическим
- Аффинность запрещает contraction $\Rightarrow$ нельзя скопировать кубит
- Weakening разрешено $\Rightarrow$ допускается игнорирование и уничтожение кубитов

Квантовый λ -исчисление

- Линейные типы для квантовых данных (qubits)
- Неограниченные типы для классических данных (bits)
- Примеры: Quipper, Q#, Silq

Аффинные типы: формальная система

Грамматика:

$$e ::= x \mid \lambda x. e \mid e_1 e_2 \mid \text{drop } e$$

Правила типизации:

$$\frac{}{x : \tau \vdash x : \tau} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x. e : \tau_1 \multimap \tau_2}$$
$$\frac{\Gamma \vdash e_1 : \tau_1 \multimap \tau_2 \quad \Delta \vdash e_2 : \tau_1}{\Gamma, \Delta \vdash e_1 e_2 : \tau_2} \quad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \text{drop } e : 1} \text{ (Drop)}$$

Отличия от линейных типов

Операция `drop` позволяет явно уничтожить ресурс (weakening)

Альтернативно: разрешить неиспользуемые переменные в контексте

Примеры в аффинной логике

Выводимо в аффинной, но не в линейной:

$$A, B \vdash A$$

(допускается игнорирование B через weakening)

Не выводимо в аффинной:

$$A \multimap (A \multimap B) \not\vdash A \multimap B$$

поскольку требуется использовать A дважды (contraction недоступна)

Сравнение термов

- Линейная: $\lambda f. \lambda x. f \ x$ — типизируется, $\lambda f. \lambda x. f \ (f \ x)$ — нет
- Аффинная: $\lambda f. \lambda x. f \ x$ — типизируется, $\lambda f. \lambda x. x$ — типизируется (drop f)
- Интуиционистская: $\lambda f. \lambda x. f \ (f \ x)$ — тоже типизируется

Свойства аффинной логики

Теорема (Консистентность)

Аффинная логика консистентна: формула $\perp$ невыводима из пустого контекста

Теорема (Нормализация)

Аффинное λ -исчисление обладает сильной нормализацией: все вычисления конечны

Теорема (Subject reduction)

Типы сохраняются при редукции: если $\Gamma \vdash e : \tau$ и $e \rightarrow e'$, то $\Gamma' \vdash e' : \tau$ для некоторого $\Gamma' \subseteq \Gamma$

Доказательство

Аналогично линейной логике, через метод логических предикатов или вложение в линейную логику с модальностью ?

Семантика аффинных типов

Категорная семантика: аффинные типы моделируются в симметрических моноидально-замкнутых категориях

- Нет диагонали (no contraction)
- Разрешено weakening (операция drop)

Интуитивная денотация: $A \multimap B$ — функции, потребляющие A ; $A \otimes B$ — отдельные ресурсы; Weakening — проекция/игнорирование

Operational semantics

Редукции аффинного λ -исчисления:

- $(\lambda x.e) v \rightarrow e[v/x]$ если x используется
- $(\lambda x.e) v \rightarrow e$ если x не используется (drop)

Замечание: строгая модель требует рассмотрения категорий с нужными свойствами

Аффинность и ownership в Rust

Система владения Rust основана на аффинных типах:

- Ownership: уникальный владелец
- Move semantics: передача = передача владения
- Drop: выход из scope = уничтожение (weakening)
- No copying: без Copy нельзя дублировать (no contraction)

Соответствие с логикой

Rust String	↔ Аффинный тип
Rust i32	↔ Тип с Copy
Rust &T	↔ Заимствование
Rust &mut T	↔ Уникальное заимствование

Заимствования и аффинность

Заимствования в Rust расширяют аффинную систему типов:

- Неизменяемое заимствование `&T`:
 - Допускается наличие нескольких одновременно (допускает contraction)
 - Только чтение
 - Пока существуют `&T`, нельзя изменить или переместить `T`
- Изменяемое заимствование `&mut T`:
 - Может быть только одно одновременно (аффинность)
 - Эксклюзивный доступ на чтение и запись
 - Нельзя иметь другие заимствования одновременно

Правило Rust

В любой момент времени: либо одно `&mut T`, либо несколько `&T`

Данное правило предотвращает состояния гонки данных (data races) на уровне системы типов

Регионы и lifetimes

Lifetimes в Rust — временные регионы существования ссылок

- Каждая ссылка имеет lifetime (регион) 'a, 'b, ...
- Lifetime указывает, как долго ссылка валидна
- Компилятор проверяет, что ссылка не переживает данные

Типизация с lifetimes:

$$\Gamma; 'a \vdash e : \&'aT$$

означает «в контексте Γ с регионом 'a, терм e имеет тип ссылки на T с временем жизни 'a»

Связь с линейной логикой

Lifetimes = линейные регионы памяти (см. работу «Linear regions are all you need»)

Region capabilities — линейные токены, контролирующие доступ к памяти

Аффинные типы и безопасность памяти

Проблемы, решаемые аффинными типами:

❶ Use-after-free:

- После `move` или `drop` переменная недоступна
- Аффинность запрещает использование после передачи владения

❷ Double-free:

- Нельзя освободить память дважды (no contraction)
- Деструктор вызывается ровно один раз

❸ Memory leaks:

- Weakening позволяет забыть ресурс (`mem::forget`)
- Но в большинстве случаев `drop` автоматический

❹ Data races:

- Правило исключительного доступа: либо `&mut T`, либо несколько `&T`
- Эксклюзивный доступ на запись (аффинность `&mut`)

Модальности в аффинной логике

Аналогично линейной логике, возможно введение модальностей для восстановления contraction

Модальность ! (разделяемость):

$$\frac{! \Gamma \vdash A}{! \Gamma \vdash ! A}$$

$$\frac{\Gamma, A \vdash B}{\Gamma, ! A \vdash B}$$

$$\frac{\Gamma, ! A, ! A \vdash B}{\Gamma, ! A \vdash B}$$

$$\frac{\Gamma \vdash B}{\Gamma, ! A \vdash B}$$

Интерпретация в Rust

- $!T \leftrightarrow$ типы с трейтом `Copy`
- Обычные типы — аффинные
- `Copy` типы — разрешают дублирование (восстанавливают contraction)

Примеры: `i32`, `bool`, `&T` — `Copy`; `String`, `Vec<T>` — аффинные

Аффинная логика и capability-based security

Capabilities (возможности) — токены доступа к ресурсам

- Идея: доступ к ресурсу требует наличия capability
- Capabilities — аффинные/линейные типы:
 - Нельзя подделать (type safety)
 - Нельзя скопировать (no contraction)
 - Допускается передача (move semantics)
 - Допускается отзыв (drop/weakening)
- Примеры:
 - FileHandle — capability на работу с файлом
 - NetworkSocket — capability на сетевое соединение
 - DatabaseConnection — capability на БД

Безопасность

Аффинность гарантирует, что ресурс не будет использован после закрытия и не будет закрыт дважды

Расширения аффинных типов

Uniqueness types (Clean)

- Гарантия отсутствия алиасинга
- Позволяет in-place mutations

Linear types with borrowing (Rust)

- Комбинация ownership + borrowing
- Borrow checker — статическая проверка

Fractional permissions

- Обобщение: доля владения вместо 0/1
- Применение в separation logic

Связь с separation logic

Separation logic — логика для рассуждений о памяти

Основные идеи

- $P * Q$ — separating conjunction (как $\otimes$ в линейной логике)
- $x \mapsto v$ — владение ячейкой памяти (линейный ресурс)
- Нельзя дублировать ownership

Применение к Rust

- Iris, RustBelt — верификация Rust через separation logic
- Система типов Rust $\approx$ автоматизированная separation logic

Вывод: аффинные типы = встроенная в язык separation logic

Fearless concurrency в Rust

Трейты для параллелизма

- Send: допускается передача между потоками (ownership)
- Sync: допускается разделение между потоками (&T)

Правило безопасности

В любой момент времени: либо `&mut T`, либо несколько `&T`

- Аффинность `&mut` $\Rightarrow$ эксклюзивный доступ
- Multiple `&T` безопасны (только чтение)

Теорема: в safe Rust отсутствуют data races на уровне модели памяти языка

Сравнение: линейная vs аффинная vs релевантная

Логика	Weak	Contr	Использование	Пример
Линейная	—	—	Ровно 1 раз	Линейные регионы
Аффинная	+	—	Не более 1 раза	Rust ownership
Релевантная	—	+	Не менее 1 раза	Обязательное логирование
Нормальная	+	+	Произвольно	STLC, Haskell

Отношения вложения:

Линейная $\subset$ Аффинная $\subset$ Нормальная

Линейная $\subset$ Релевантная $\subset$ Нормальная

Каждая более слабая логика допускает больше доказательств (меньше ограничений)

Промежуточное резюме: аффинные типы

Аффинная логика

- Разрешает weakening (допускается неиспользование)
- Запрещает contraction (нельзя копировать)
- Использование: не более одного раза

Практика: Rust ownership

- Move semantics — передача владения
- Borrowing — временное использование
- Гарантии безопасности памяти без GC

Ключевое отличие: линейность (ровно 1) vs аффинность (не более 1)

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов**
- 7 Заключение

Субструктурные типы в языках программирования

- Rust: аффинная система типов
 - Move semantics — значения перемещаются, а не копируются
 - Трейт Copy восстанавливает contraction
 - Lifetime система основана на линейных регионах
- Cyclone: безопасный C с регионами
 - Unique pointers — аффинные типы
 - Region-based memory management
 - Статическая проверка использования памяти
- Session types: линейные типы для протоколов
 - Каждое сообщение отправляется/получается ровно один раз
 - Гарантия отсутствия дедлоков
 - Реализации: Links, Singularity OS

Сессионные типы и линейная логика

Соответствие: процессы $\leftrightarrow$ доказательства, каналы $\leftrightarrow$ формулы

Линейная логика	Процессы
$A \otimes B$	Отправка A, затем B
$A \oplus B$	Выбор: отправить A или B
$A \& B$	Предложение выбора между A и B
$A \multimap B$	Получить A, отправить B
$!A$	Репликация (сервер, неограниченное использование)
$?A$	Репликация (клиент, неограниченное использование)

Примечание: Здесь $!A$ и $?A$ — экспоненциалы LL (репликация), не конструкторы `send/recv`

Правило (cut):

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : A^\perp}{(\nu x^A y)(P \mid Q) \vdash \Gamma, \Delta}$$

В дисциплине `session types` при дуальности концов и корректной композиции (`well-typed processes`) достигаются `progress` и `deadlock-freedom`; линейность каналов — необходимая часть этой дисциплины, но не единственная.

План лекции

- 1 Мотивация
- 2 Линейная логика
- 3 Линейная логика: формальная система
- 4 Применение линейных типов
- 5 Аффинная логика
- 6 Применения субструктурных типов
- 7 Заключение

Субструктурные логики и типы:

- 1 Структурные правила (exchange, weakening, contraction) определяют, как манипулировать контекстом
- 2 Линейная логика — логика ресурсов без weakening и contraction
 - Мультипликативные связки: $\otimes, \multimap, 1, \perp$
 - Аддитивные связки: $\&, \oplus, \top, 0$
 - Экспоненциалы: $!, ?$ — восстанавливают структурные правила
 - Дуальность: $(A \otimes B)^\perp = A^\perp \wp B^\perp, (A \oplus B)^\perp = A^\perp \& B^\perp$
- 3 Аффинная логика — добавляет weakening (at most once)
- 4 Применения: управление памятью (Rust, Cyclone), сессионные типы, параллельные вычисления
- 5 Соответствие Карри-Ховарда обобщается на субструктурные системы

Вывод: субструктурные типы кодируют инварианты использования ресурсов в системе типов

- Jean-Yves Girard. Linear Logic
- Philip Wadler. Linear Types Can Change the World
- Simon Gay, Vasco Vasconcelos. Linear Type Theory for Asynchronous Session Types
- The Rust Programming Language. Ownership and Borrowing
- Dan Grossman, Greg Morrisett, et al. Region-Based Memory Management in Cyclone
- Frank Pfenning, Dennis Griffith. Polarized Substructural Session Types